

MNIST 디퓨전 모델 구현 가이드

개요

MNIST 손글씨 숫자 데이터셋을 사용한 간단한 디퓨전 모델(Diffusion Model)의 구현을 설명합니다. 디퓨전 모델은 이미지 생성 AI의 핵심 기술로, 노이즈를 점진적으로 제거하면서 새로운 이미지를 생성합니다.

1. 초기 설정 및 라이브러리 импорт

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)
```

설명:

- PyTorch와 관련 라이브러리를 импорт합니다
- GPU 사용 가능 여부를 확인하고 디바이스를 설정합니다
- GPU가 있으면 "cuda", 없으면 "cpu"를 사용합니다

2. MNIST 데이터셋 준비

```
transform = transforms.ToTensor()
ds = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
loader = DataLoader(ds, batch_size=128, shuffle=True)
```

설명:

- transform:** 이미지를 텐서로 변환하는 변환기
- datasets.MNIST:** MNIST 손글씨 숫자 데이터셋 로드
 - root="./data": 데이터 저장 경로
 - train=True: 학습용 데이터 사용
 - download=True: 데이터가 없으면 자동 다운로드
- DataLoader:** 배치 단위로 데이터를 제공
 - batch_size=128: 한 번에 128개 이미지 처리
 - shuffle=True: 데이터를 무작위로 섞음

3. 디퓨전 스케줄 설정

```
T = 1000
beta = torch.linspace(1e-4, 0.02, T).to(device)
alpha = 1 - beta
alpha_bar = torch.cumprod(alpha, dim=0)
```

설명:

- **T = 1000**: 총 디퓨전 스텝 수 (노이즈 추가/제거 단계)
- **beta**: 각 타임스텝에서의 노이즈 스케일
 - 0.0001부터 0.02까지 선형적으로 증가
 - 초반에는 적은 노이즈, 후반에는 많은 노이즈 추가
- alpha**: $1 - \beta_t$ (원본 이미지 보존 비율)
- alpha_bar**: alpha의 누적곱
 - t 스텝까지의 전체 노이즈 영향을 계산하는 데 사용

수식:

- $\alpha_t = 1 - \beta_t$
- $\bar{\alpha}_t = \prod_{i=1 \rightarrow t} \alpha_i$

4. UNet 아키텍처 구현

4.1 Block 클래스

```
class Block(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, padding=1),
            nn.GroupNorm(1, out_c),
            nn.SiLU(),
            nn.Conv2d(out_c, out_c, 3, padding=1),
            nn.GroupNorm(1, out_c),
            nn.SiLU(),
        )

    def forward(self, x):
        return self.conv(x)
```

설명:

- UNet의 기본 빌딩 블록
- **구성 요소**:
 - Conv2d: 3x3 합성곱 레이어 (특징 추출)
 - GroupNorm: 정규화 레이어 (학습 안정화)
 - SiLU: 활성화 함수 (비선형성 추가)

두 번의 합성곱-정규화-활성화 과정을 거칩니다

4.2 UNet 클래스

```
class UNet(nn.Module):
    def __init__(self):
        super().__init__()
```

```

self.down1 = Block(1, 32)
self.down2 = Block(32, 64)
self.down3 = Block(64, 128)

self.pool = nn.MaxPool2d(2)

self.up1 = Block(128 + 64, 64)
self.up2 = Block(64 + 32, 32)

self.final = nn.Conv2d(32, 1, 1)

def forward(self, x):
    # Encoder (인코더 - 다운샘플링)
    x1 = self.down1(x)
    x2 = self.down2(self.pool(x1))
    x3 = self.down3(self.pool(x2))

    # Decoder (디코더 - 업샘플링)
    u1 = F.interpolate(x3, scale_factor=2, mode="nearest")
    u1 = self.up1(torch.cat([u1, x2], dim=1))

    u2 = F.interpolate(u1, scale_factor=2, mode="nearest")
    u2 = self.up2(torch.cat([u2, x1], dim=1))

    return self.final(u2)

```

설명:

- **Encoder (인코더):** 이미지 크기를 줄이면서 특징을 추출
 - down1: $1 \rightarrow 32$ 채널 (28x28)
 - down2: $32 \rightarrow 64$ 채널 (14x14, 풀링 후)
 - down3: $64 \rightarrow 128$ 채널 (7x7, 풀링 후)
- Decoder (디코더):** 이미지 크기를 복원하면서 노이즈 예측
 - up1: $128+64 \rightarrow 64$ 채널 (14x14, 업샘플링 + 스킵 연결)
 - up2: $64+32 \rightarrow 32$ 채널 (28x28, 업샘플링 + 스킵 연결)
- 스킵 연결:** 인코더의 특징을 디코더에 전달 (torch.cat)
 - 세부 정보 보존에 도움
- 최종 레이어:** $32 \rightarrow 1$ 채널 (예측된 노이즈)

5. 모델 및 최적화 설정

```
model = UNet().to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.MSELoss()
```

설명:

- **model**: UNet 모델을 생성하고 GPU/CPU로 이동
- **opt**: Adam 옵티마이저 (학습률 0.0001)
- **criterion**: 평균제곱오차(MSE) 손실 함수
 - 예측한 노이즈와 실제 노이즈의 차이를 계산

6. Forward Diffusion (순방향 디퓨전)

```
def q_sample(x0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)

    sqrt_ab = torch.sqrt(alpha_bar[t]).view(-1, 1, 1, 1)
    sqrt_1_ab = torch.sqrt(1 - alpha_bar[t]).view(-1, 1, 1, 1)

    return sqrt_ab * x0 + sqrt_1_ab * noise
```

설명:

- **목적**: 깨끗한 이미지 x_0 에 노이즈를 추가하여 x_t 생성
- **입력**:
 - x_0 : 원본 이미지
 - t : 타임스텝 (0~999)
 - noise : 가우시안 노이즈 (없으면 자동 생성)

수식 구현:

$$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \varepsilon$$

- sqrt_ab : $\sqrt{\alpha_t}$ (원본 보존 비율)
- sqrt_1_ab : $\sqrt{(1-\alpha_t)}$ (노이즈 비율)

의미:

- t 가 클수록 노이즈가 많아짐
 - $t=0$: 거의 원본
 - $t=999$: 거의 순수 노이즈
-

7. 모델 학습

```
print("Training...")

for epoch in range(20):
    for x0, _ in loader:
        x0 = x0.to(device)
        b = x0.shape[0]

        t = torch.randint(0, T, (b,), device=device)
        noise = torch.randn_like(x0)

        # forward: x_t 생성
        x_t = q_sample(x0, t, noise)

        # 모델이 예측하는 것은 noise( $\epsilon$ )
        eps_pred = model(x_t)

        loss = criterion(eps_pred, noise)

        opt.zero_grad()
        loss.backward()
        opt.step()

    print(f"[epoch {epoch}] loss = {loss.item():.4f}")
```

설명:

- 학습 과정:
 1. 원본 이미지 x_0 로드
 2. 랜덤 타임스텝 t 선택 (0~999)
 3. 랜덤 노이즈 생성
 4. q_sample 로 노이즈가 추가된 x_t 생성
 5. 모델이 x_t 를 보고 노이즈 예측
 6. 예측 노이즈와 실제 노이즈 차이로 손실 계산
 7. 역전파로 모델 가중치 업데이트

핵심 아이디어:

- 모델은 "노이즈가 섞인 이미지를 보고 원래 노이즈를 맞추는" 법을 학습
 - 다양한 타임스텝에서 학습하여 모든 노이즈 레벨에 대응
-

8. Reverse Sampling (역방향 샘플링)

8.1 단일 스텝 샘플링

```
@torch.no_grad()
def p_sample(x, t_int):
    """
    t_int = int (Python int)
    """
    t = torch.tensor([t_int], device=device).long()
    beta_t = beta[t_int]
    alpha_t = alpha[t_int]
    alpha_bar_t = alpha_bar[t_int]

    eps = model(x)

    mean = (1 / torch.sqrt(alpha_t)) * (
        x - beta_t / torch.sqrt(1 - alpha_bar_t) * eps
    )

    if t_int == 0:
        return mean

    z = torch.randn_like(x)
    sigma = torch.sqrt(beta_t)
    return mean + sigma * z
```

설명:

- 목적: x_t 에서 x_{t-1} 로 한 단계 노이즈 제거
- 과정:
 1. 모델이 현재 이미지의 노이즈 예측 (eps)
 2. 평균(mean) 계산:
 3. $mean = (1/\sqrt{\alpha_t}) \cdot (x_t - \beta_t/\sqrt{(1-\bar{\alpha}_t)} \cdot \epsilon)$
 - 4.
 5. $t=0$ 이면 평균만 반환 (최종 이미지)
 6. $t \neq 0$ 이면 약간의 랜덤성 추가:
 7. $x_{t-1} = mean + \sqrt{\beta_t} \cdot z$
 - 8.
 9. (z 는 가우시안 노이즈)

8.2 전체 샘플링 과정

```
@torch.no_grad()
def sample(n=16):
    x = torch.randn(n, 1, 28, 28).to(device)
    for t_int in reversed(range(T)):
        x = p_sample(x, t_int)
    return x
```

설명:

- 목적: 순수 노이즈에서 시작하여 이미지 생성
- 과정:

1. 랜덤 노이즈로 시작 (x_T)
2. $T-1$ 부터 0까지 역순으로 반복
3. 각 스텝마다 p_{sample} 로 노이즈 제거
4. 최종적으로 깨끗한 이미지 x_0 생성

샘플링 흐름:

순수 노이즈 ($t=999$)

↓ p_{sample}

약간 덜 노이즈 ($t=998$)

↓ p_{sample}

...

↓ p_{sample}

희미한 숫자 모양 ($t=500$)

↓ p_{sample}

...

↓ p_{sample}

선명한 숫자 ($t=0$)

9. 이미지 생성 및 시각화

```
print("Sampling images...")

samples = sample(8).cpu()

fig, axs = plt.subplots(1, 8, figsize=(12, 2))
for i in range(8):
    axs[i].imshow(samples[i, 0], cmap="gray")
    axs[i].axis("off")
plt.show()
```

설명:

- 8개의 새로운 MNIST 숫자 이미지 생성
- CPU로 이동 후 matplotlib로 시각화
- 그레이스케일로 표시
- 축 레이블 제거하여 깔끔하게 표시

10. 디퓨전 모델 동작 원리 요약

학습 단계 (Training)

1. 실제 MNIST 이미지를 가져옴
2. 랜덤 타임스텝 t 를 선택
3. 이미지에 노이즈를 추가하여 x_t 생성
4. 모델이 노이즈를 예측하도록 학습
5. 예측 노이즈와 실제 노이즈의 차이를 최소화

생성 단계 (Sampling)

1. 순수 노이즈에서 시작
2. 모델이 노이즈를 예측
3. 예측한 노이즈를 제거
4. 1000번 반복하여 점진적으로 깨끗한 이미지 생성

핵심 개념

- **Forward Process:** 이미지 $\rightarrow$ 노이즈 (학습 데이터 생성)
- **Reverse Process:** 노이즈 $\rightarrow$ 이미지 (생성)
- **UNet:** 노이즈를 예측하는 신경망
- **점진적 디노이징:** 한 번에 모든 노이즈를 제거하는 것이 아니라 1000단계에 걸쳐 천천히 제거

11. 주요 하이퍼파라미터

파라미터	값	설명
T	1000	디퓨전 스텝 수
beta 범위	0.0001 ~ 0.02	노이즈 스케줄
batch_size	128	배치 크기
learning_rate	0.0001	학습률
epochs	20	학습 에포크
이미지 크기	28x28	MNIST 기본 크

12. 개선 가능한 부분

- 1. 타임스텝 조건화: 현재 UNet은 타임스텝 정보를 사용하지 않음
 - 타임스텝 임베딩 추가 권장
- 더 깊은 UNet: 현재는 교육용 간단한 버전
 - 레이어 추가로 성능 향상 가능
- 학습 에포크: 20 에포크는 데모용
 - 실제로는 수십~수백 에포크 필요
- 스케줄러: 고정된 beta 대신
 - Cosine 스케줄 등 개선된 방법 사용 가능
- 조건부 생성: 특정 숫자 지정하여 생성
 - 클래스 조건화 추가 가능

참고 자료

- DDPM 논문: "Denoising Diffusion Probabilistic Models" (Ho et al., 2020)
- 원리: 열역학의 확산 과정에서 영감
- 응용: Stable Diffusion, DALL-E 등의 기반 기술

이 코드는 디퓨전 모델의 핵심 개념을 이해하기 위한 교육용 구현입니다. 실제 프로덕션에서는 더 복잡한 아키텍처와 최적화가 필요합니다.